

Overview

You and your friends have varying hobbies and interests and you, as a developer, decided to create an application for your friends. Your previous work on hobby sites now proved useful as you have the backing of your local cultural community to organize larger hubs where people of varying interests and activities are present and contributed by all. Dubbed as “Common Grounds”, your application will serve a larger audience and more complex features. After a bit more work, you decided to expand this site to include user inputs and functionalities to expand your user base and cater to the applications’ needs.

Specifications

1. Update the midterm common grounds site according to the specifics below.
2. Each app should have the described list, detail, create, and update views, along with the appropriate admin panels.
3. User management should be implemented (register, login, logout). Utilize the default Django URLs for authentication as described in Lecture 16 - Managing Users. User inputs should be attributed to the logged in user. All Create and Update views require logins unless otherwise stated.
4. All Create and Update views should go to the created/updated object’s Detail View.
5. Each model should have human-readable representations of model instances.
6. Create a homepage with the URL at the project’s root (localhost:8000) that can be used to link to all the apps. Use this, also, as the redirect page for logins.
7. There should be ONE (1) advanced requirement that is implemented.
8. Make sure that your applications have the necessary data and objects to present the different functionalities required.

Rubrics

Deductions will be imposed for violations which include, but are not limited to, clean coding principles and proper use of Git.

Submission Details

1. Choose one member of your group to create a Github repository for your group, which is to be named **commongrounds-groupnumber**.
2. The initial commit will just contain a README.txt and the .gitignore file. The README should contain the name of the group members, in alphabetical order according to last name.
3. Each app should be developed on separate branches, branched out after creating the Django project. These branches are to be merged BEFORE submission of the link.
4. You are to submit the **Railway application link** of your project. Make sure that it is visible to the public.
5. Expected completion date is on **7 May 2026, 11:59PM**.

IMPORTANT:

- Follow file naming conventions.
- Follow submission procedures.

Detailed Descriptions

Accounts app

- Patterns: Decorator / Mixin
- app_name: accounts
- URLs:
 - As described in Lecture 16 - Managing Users
 - Update view (/accounts/<username>)
- Models:
 - Profile
 - An extension of User model
 - Display name - max length is 63 characters
 - Email address - email field
- Views:
 - Profile Update View:
 - It should allow updates to the Display name only.
 - This should only be accessible to logged-in users.
- Advanced Requirements:
 - Create a RoleRequiredMixin that can be applied to any CBV to restrict access based on the Profile's role. It should redirect unauthorized users to a permission-denied page or the login page.
 - Create a @role_required function decorator that provides the same behavior for FBVs.
 - Both must accept the required role as an argument (e.g., @role_required("Market Seller")).
 - Apply the mixin to the Product Create View and Event Create View. Apply the decorator to at least one FBV of your choice.
 - Both implementations must live in accounts/mixins.py and accounts/decorators.py respectively.

Merchandise Store app

- Patterns: Strategy + Observer
- app_name: merchstore
- URLs:
 - Product List View: /merchstore/items
 - Product Detail View: /merchstore/item/1
 - Product Create View : /merchstore/item/add
 - Product Update View: /merchstore/item/1/edit
 - Cart View: /merchstore/cart
 - Transactions List View: /merchstore/transactions
- Models:
 - ProductType
 - Name - max length is 255 characters
 - Description - text field
 - Product Types should be sorted by name in ascending order
 - Product
 - Name - max length is 255 characters
 - Product Type - foreign key to ProductType, set to NULL when deleted, determined by developers and cannot be modified by regular users
 - Owner - foreign key to Profile, model deletion is cascaded
 - Product Image - image field
 - Description - text field
 - Price - positive number with two decimal places
 - Stock - whole number
 - Status - character field with the following options:
 - Available (This should be the default value)
 - On sale
 - Out of stock (when stock is 0)
 - Products should be sorted by name in ascending order
 - Transaction
 - Buyer - foreign key to Profile, set to NULL when deleted
 - Product - foreign key to Product, model deletion is cascaded
 - Amount - natural number which is the amount of Product to buy
 - Status - character field with the following options:
 - On cart
 - To Pay
 - To Ship
 - To Receive
 - Delivered
 - Created On - datetime field, only gets set when the model is created
- Views:
 - Product List View
 - List ALL products in the system, regardless of who posted and status.
 - When logged in, products that are posted for sale by the logged-in user are in a separate group. They should also be removed from the All Products list.

- User's Products should come before All Products.
 - In this view, there should be a link that will lead to the creation of a product.
- Product Detail View
 - Users should not be able to purchase their own products.
 - There should be a Form that is rendered on this view, whose model is the Transaction model.
 - This Transaction form should be handled by the FBV/CBV that is handling the rendering of the page.
 - When the transaction form is submitted and the form fields are being processed, the corresponding product's stock should be changed accordingly.
 - If the Transaction form has all the necessary details and is saved, the user will be redirected according to the login status. If the user is authenticated, then they will be redirected to the CartView. Otherwise, they will be redirected to the login page. Once logged-in, only then can the transaction be saved.
 - If the Product's stock is 0, the buy button should not be clickable.
(Hint: you can conditionally add an HTML attribute based on information that can derived from the available context objects)
 - In this view, if the Product's owner is the logged-in user, there should be an edit link that will lead to the update view.
- Product Create View
 - This view is only available if the Profile has the role "Market Seller"
 - This should only be accessible to logged-in users.
 - All fields should be available, and the Owner field is set to the logged in user that is not editable.
 - Product Type and Status should be a dropdown.
- Product Update View
 - This view is only available if the Profile has the role "Market Seller".
 - This should only be accessible to logged-in users.
 - It should allow updates of all fields except the Owner field.
 - When the product's stock is 0, status is automatically set as "Out of Stock", otherwise it's set as "Available".
- Cart View
 - Cart View shows all the Transactions made by the logged-in user as the buyer.
 - The transactions should be categorized by the Product's owner, similar in layout to the Product List View
 - This should only be accessible to logged-in users.
- Transaction List View
 - Transaction List View shows all the Transactions made by the logged-in user as the seller.
 - The transactions should be categorized by the Product's buyer, similar in layout to the Product List View.
 - This should only be accessible to logged-in users.
- Advanced Requirements:
 - Strategy: Create a BaseTransactionStrategy class with an execute(request, product, form) method. Implement two concrete strategies:

- `AuthenticatedPurchaseStrategy` – saves the transaction, deducts stock, updates product status, and redirects to the Cart View.
- `GuestPurchaseStrategy` – stores the pending transaction data in the session and redirects to the login page. Once logged in, the transaction is completed.
- The Product Detail View must select and call the appropriate strategy based on the user's authentication status. Strategy classes must live in `merchstore/strategies.py`.
- Observer: Use a Django `post_save` signal on the Transaction model. When a Transaction is created, the signal handler must automatically deduct the transaction amount from the corresponding Product's stock and update the Product's status if stock reaches 0. Signal handlers must live in `merchstore/signals.py` and be connected in `merchstore/apps.py`.

Local Events app

- Pattern: Template Method
- app_name: localevents
- URLs:
 - Event List View: /localevents/events
 - Event Detail View: /localevents/event/1
 - Event Create View: /localevents/event/add
 - Event Update View: /localevents/event/1/edit
 - Event Signup View: /localevents/event/1/signup
- Models:
 - EventType
 - Name - max length is 255 characters
 - Description - text field
 - Types should be sorted by name in ascending order
 - Event
 - Title - max length is 255 characters
 - Category - foreign key to EventType, sets to NULL when deleted, determined by developers and cannot be modified by regular users
 - Organizer - many-to-many relationship to Profile, sets to NULL when deleted
 - Event Image - image field
 - Description - text field
 - Location - character field
 - Start time - datetime field
 - End time - datetime field
 - Event capacity - natural number
 - Status - character field with the following statuses:
 - Available
 - Full
 - Done
 - Cancelled
 - Created On - datetime field, only gets set when the model is created
 - Updated On - datetime field, always updates on last model update
 - Events should be sorted by the date it was created, in descending order
 - EventSignup
 - Event - foreign key to Event, model deletion is cascaded
 - UserRegistrant - foreign key to Profile, model deletion is cascaded, set when registrant is logged in user
 - NewRegistrant - character field, set when registrant is not logged in
- Views
 - Event List View
 - List ALL events in the system, regardless of who posted and status.
 - When logged in, there should be groups for the following book entries, displayed first, and removed from the All Events list:
 - Events created
 - Events signed up

- User's events should come before All Events.
 - In this view, there should be a link that will lead to the creation of an event.
- Event Detail View
 - Users should not be able to sign up on their own events.
 - There should be a button that is rendered on this view that allows you to sign up for the event. If you are logged-in, the sign up is automatic. Else, you should be redirected to a view with a form to fill up your name to sign up.
 - If the number of event signups is equal or more than the event capacity, the sign up button should not be clickable.
(Hint: you can conditionally add an HTML attribute based on information that can be derived from the available context objects)
 - In this view, if the Event's owner is the logged-in user, there should be an edit link that will lead to the update view.
- Event Create View
 - This view is only available if the Profile has the role "Event Organizer".
 - This should only be accessible to logged-in users.
 - All fields should be available, and the Organizer field is set to the logged in user that is not editable.
 - Category and Status should be a dropdown.
- Event Update View
 - This view is only available if the Profile has the role "Event Organizer".
 - This should only be accessible to logged-in users.
 - It should allow updates of all fields except the Organizer field.
 - When the event signups are more than or equal to the event capacity, status is automatically set as "Full", otherwise it's set as "Available".
- Event Signup Form
 - This view is only available when you are not logged in and clicked the sign up button from the Detail View.
 - The form should show the event information.
 - The only required input from the user is their name.
- Advanced Requirements:
 - Create an abstract base CBV BaseSignupView in localevents/views.py that defines the following Template Method skeleton in its post() method:
 - check_capacity(event) — returns True if signup is still allowed
 - check_ownership(event, user) — returns True if the user is not the organizer
 - create_signup(event, user) — creates the signup record
 - get_redirect_url(event) — returns the URL to redirect to after signup
 - Each step must be a separate overridable method.
 - Implement a concrete EventSignupView that subclasses BaseSignupView with the logic specific to Local Events.
 - At least one method (e.g., get_redirect_url) must be demonstrably overridden in a subclass or test case to show the pattern working.

Book Club app

- Pattern: Factory Method
- app_name: bookclub
- URLs:
 - Book List View: /bookclub/books
 - Book Detail View: /bookclub/book/1
 - Book Create View: /bookclub/book/add
 - Book Update View: /bookclub/book/1/edit
 - Book Borrow View: /bookclub/book/1/borrow
- Models:
 - Genre
 - Name - max length is 255 characters
 - Description - text field
 - Genres should be sorted by name in ascending order
 - Book
 - Title - max length is 255 characters
 - Genre - foreign key to Genre, sets to NULL when deleted, determined by developers and cannot be modified by regular users
 - Contributor - foreign key to Profile, sets to NULL when deleted
 - Author - character field
 - Synopsis - text field
 - Publication year - integer field
 - Available to borrow - Boolean field
 - Created On - datetime field, only gets set when the model is created
 - Updated On - datetime field, always updates on last model update
 - Books should be sorted by the date it was published, in descending order
 - BookReview
 - UserReviewer - foreign key to Profile, model deletion is cascaded, set when reviewer is logged in
 - AnonReviewer - text field, set when reviewer is not logged in
 - Book - foreign key to Book, model deletion is cascaded
 - Title - character field
 - Comment - text field
 - Bookmark
 - Profile - foreign key to Profile, model deletion is cascaded
 - Book - foreign key to Book, model deletion is cascaded
 - Date bookmarked- date field
 - Borrow
 - Book - foreign key to Book, model deletion is cascaded
 - Borrower - foreign key to Profile, model deletion is cascaded, set when borrower is logged in
 - Name - character field, set when borrower is not logged in
 - Date borrowed - date field
 - Date to return - date field
- Views

- Book List View
 - List ALL books in the system, regardless of who posted and status.
 - When logged in, there should be groups for the following book entries, displayed first, and removed from the All Books list:
 - Books contributed to the system
 - Books bookmarked
 - Books reviewed
 - In this view, there should be a link that will lead to the creation of a book.
- Book Detail View
 - There should be a display whether the book is available to be borrowed.
 - There should be a button that allows you to bookmark the book. The number of bookmarks on the book should also be displayed.
 - There should be a Form that is rendered on this view that allows you to review the book. When the user is logged in, the review is automatically assigned to the Profile with the associated display name. Else, the display name should be "Anonymous".
 - There should be a button to borrow a book that will lead to the borrow view.
 - In this view, if the Book's contributor is the logged-in user, there should be an edit link that will lead to the update view.
 - The list of book reviews should be shown.
- Book Create View
 - This view is only available if the Profile has the role "Book Contributor".
 - This should only be accessible to logged-in users.
 - All fields should be available, and the Contributor field is set to the logged in user that is not editable.
 - Genre should be a dropdown.
- Book Update View
 - This view is only available if the Profile has the role "Book Contributor".
 - This should only be accessible to logged-in users.
 - It should allow updates of all fields except the Contributor field.
- Book Borrow View
 - The form should show the book information.
 - If you are logged in, the name entry is automatically filled, else the only required input from the user is their name.
 - The user can set the date they can borrow the book, but the return date should always be two weeks from the borrowing date, including weekends and holidays.
- Advanced Requirements:
 - Create a BookFormFactory class in bookclub/forms.py with a get_form(context) class method. It must return different form classes based on the context:
 - "review" — returns a BookReviewForm with Reviewer pre-set to the logged-in user and not editable
 - "contribute" — returns a BookContributeForm with Contributor pre-set to the logged-in user and not editable
 - "update" — returns a BookUpdateForm that excludes the Contributor field

- The Book Detail View and Book Create View must obtain their form classes exclusively through the factory. Direct instantiation of form classes in views is not allowed.

DIY Projects app

- Pattern: Repository
- app_name: diyprojects
- URLs:
 - Project List View: /diyprojects/projects
 - Project Detail View: /diyprojects/project/1
 - Project Create View: /diyprojects/project/add
 - Project Update View: /diyprojects/project/1/edit
- Models:
 - ProjectCategory
 - Name - max length is 255 characters
 - Description - text field
 - Categories should be sorted by name in ascending order
 - Project
 - Title - max length is 255 characters
 - Category - foreign key to ProjectCategory, sets to NULL when deleted
 - Creator - foreign key to Profile, sets to NULL when deleted
 - Description - text field
 - Materials - text field
 - Steps - text field
 - Created On - datetime field, only gets set when the model is created
 - Updated On - datetime field, always updates on last model update
 - Posts should be sorted by the date it was created, in descending order
 - Favorite
 - Project - foreign key to Project, model deletion is cascaded
 - Profile - foreign key to Profile, model deletion is cascaded
 - Date favorited - date field
 - Project status - character field with the following options:
 - Backlog
 - To-Do
 - Done
 - ProjectReview
 - Reviewer - foreign key to Profile, model deletion is cascaded
 - Comment - text field
 - Image - image field
 - ProjectRating
 - Profile - foreign key to Profile, model deletion is cascaded
 - Score - integer field from 1 to 10
- Views
 - Project List View
 - List ALL projects in the system, regardless of who posted and status.
 - When logged in, there should be groups for the following project entries, displayed first, and removed from the All Books list:
 - Projects created
 - Projects favorited

- Projects reviewed
 - In this view, there should be a link that will lead to the creation of a project.
 - Project Detail View
 - There should be an average of the project rating scores in the view. There should be a Form that is rendered on this view that allows you to send a project rating. Submitting a project rating is only available for logged in users.
 - There should be a button that allows you to favorite the book. The number of favorites on the project should also be displayed.
 - There should be a Form that is rendered on this view that allows you to review the project. Submitting a project review is only available for logged in users.
 - In this view, if the Project's creator is the logged-in user, there should be an edit link that will lead to the update view.
 - The list of project reviews should be shown.
 - Project Create View
 - This view is only available if the Profile has the role "Project Creator".
 - This should only be accessible to logged-in users.
 - All fields should be available, and the Owner field is set to the logged in user that is not editable.
 - Category should be a dropdown.
 - Project Update View
 - This view is only available if the Profile has the role "Project Creator".
 - This should only be accessible to logged-in users.
 - It should allow updates of all fields except the Creator field.
- Advanced Requirements:
 - Create a ProjectRepository class in diyprojects/repositories.py. It must implement the following methods – direct ORM calls (e.g., Project.objects.all()) are only permitted inside this class, not in views:
 - get_all() – returns all projects
 - get_by_category(category_name) – returns projects filtered by category name
 - get_recent(n) – returns the n most recently created projects
 - get_by_id(id) – returns a single project by its primary key
 - Both views must obtain their data exclusively through an instance of ProjectRepository.
 - The repository must be instantiated once per view, not per method call.

Commission requests app

- Pattern: Facade / Service Layer
- app_name: commissions
- URLs:
 - List View: /commissions/requests
 - Detail View: /commissions/request/1
 - Create View: /commissions/request/add
 - Update View: /commissions/request/1/edit
- Models:
 - CommissionType
 - Name - max length is 255 characters
 - Description - text field
 - Types should be sorted by name in ascending order
 - Commission
 - Title - max length is 255 characters
 - Description - text field
 - Type - foreign key to CommissionType, set to NULL when deleted
 - Maker - foreign key to Profile, model deletion is cascaded
 - People Required - should be whole number
 - Status - character field with the following options:
 - Open (This should be the default value)
 - Full
 - Created On - datetime field, only gets set when the model is created
 - Updated On - datetime field, always updates on last model update
 - Commissions should be sorted by the date it was created, in ascending order
 - Job
 - Commission - foreign key to Commission, model deletion is cascaded
 - Role - max length is 255 characters
 - Manpower Required - should be whole number
 - Status - character field with the following options:
 - Open (This should be the default value)
 - Full
 - Jobs should be sorted by status (Open > Full), manpower required, in descending order, then role, in ascending order
 - JobApplication
 - Job - foreign key to Job, model deletion is cascaded
 - Applicant - foreign key to Profile, model deletion is cascaded
 - Status - character field with the following options:
 - Pending (default)
 - Accepted
 - Rejected
 - Applied On - datetime field, only gets set when the model is created
 - Should be sorted by status (Pending first, then Accepted, then Rejected), then Applied On, in descending order
- Views

- Commission List View:
 - List ALL commissions in the system, regardless of poster and status. The commissions should follow this sorting schema:
 - Status (Open > Full > Completed > Discontinued)
 - Created On, most recent is shown first
 - When logged in, there should be groups for the following commission entries, displayed first, and removed from the All Commissions list:
 - Commissions the logged-in user has created
 - Commissions the logged-in user has applied to
 - In this view, there should be a link that will lead to the creation of a commission.
- Commission Detail View
 - This view should show the commission information and the list of Jobs needed for the Commission.
 - A sum of manpower required, and open manpower (difference between sum of manpower and accepted signees) from the jobs should be shown.
 - When logged in, the “Apply to Job” button to the Job should be open. This is essentially a JobApplication form, similar to the MerchStore Transaction form
 - When the number of JobApplication entries with “Accepted” status is greater than or equal to the Job’s manpower required, the “Apply to Job” button should not be clickable.
 - In this view, if the Commission's owner is the logged-in user, there should be an edit link that will lead to the update view.
- Commission Create View:
 - This view is only available if the Profile has the role “Commission Maker”.
 - This should only be accessible to logged-in users.
 - All fields should be available, including the corresponding Job objects, and the Maker field is set to the logged in user that is not editable.
 - Status field should be a dropdown.
- Commission Update View:
 - This view is only available if the Profile has the role “Commission Maker”.
 - This should only be accessible to logged-in users.
 - It should allow updates of all fields except the Maker field.
 - When all the commission’s jobs’ statuses are “Full”, the commission’s status should be set to “Full”.
- Advanced Requirements:
 - Create a CommissionService class in commissions/services.py. Views must not contain business logic directly — all coordination between models must go through this service. It must implement the following methods:
 - create_commission(author, data, jobs_data) — creates the Commission and all associated Job entries atomically
 - apply_to_job(applicant, job) — creates a JobApplication, checking that the applicant has not already applied and that the job is not full
 - sync_commission_status(commission) — checks all related Jobs and sets the commission status to "Full" if all jobs are full; called on every Commission update

- `get_commission_summary(commission)` — returns a dictionary with `total_manpower` and `open_manpower` computed from all related Jobs and their accepted applications
- All four methods must be used in the appropriate views or signals.
- `create_commission` must use `transaction.atomic()` to ensure the Commission and its Jobs are created together or not at all.

Bonus

- Apply CSS to your templates (Tailwind, Bootstrap, etc.) that create a consistent visual design identity across **ALL APPLICATIONS**.
- Create a dashboard view for the Profile application that shows the logged-in user's all user-made objects across all applications.
 - List of products sold
 - List of events organized
 - List of books contributed
 - List of projects created
 - List of commissions created
- Deploying on DigitalOcean, instead of Railway
- Any other additions to the project that makes sense in the context of the project can be considered.